



**R V College of Engineering**  
**Department of Computer Science and Engineering**  
**Improvement CIE: Scheme**

**Subject :  
(Code)**

**COMPILER DESIGN (CS363AI)**

**Semester : 6<sup>th</sup> BE**

**Date : JUNE 2025**

**Duration : 120 minutes**

**Staff : DD/SB/SNM/SMS/PK**

**Name :**

**USN :**

**Section :  
A,B,C,D,ISE**

| Si.<br>No | Part A<br>Quiz Questions                                                                                                                                                                       | M | BT | Co |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|----|----|
| 1.1       | Short circuit codes are the Boolean operators &&,    and ! translations into jumps.---1 mark<br>Example --- 1 mark                                                                             | 2 |    |    |
| 1.2       | Consider the grammar below:<br>S->SS+   SS*   a<br>If I = {S->S.S+} compute Closure of (I).<br>Closure(I)={ S->S.S+, S->.SS+, S->.SS*, S->.a}                                                  | 2 |    |    |
| 1.3.      | If x<100 goto L2<br>goto L3<br>L3: if x>200 goto L4<br>goto L1<br>L4: if x!=y goto L2<br>goto L1<br>L2: x=2<br>L1:                                                                             | 2 |    |    |
| 1.3       | Construct a DAG for the following basic block:<br>T1=A+B<br>T2=C+D<br>T3=E-T2<br>T4=T1-T3                                                                                                      | 2 |    |    |
| 1.4       | Backpatch(p,i)                                                                                                                                                                                 | 1 |    |    |
| 1.5       | Write the backpatching SDT for the following production:<br>B→B1 && M B2<br>{B.truelist= B2.truelist;<br>B.falselist = merge(B1.falselist,B2.falselist);<br>Backpatch(B1. Truelist, M.instr);} | 1 |    |    |
| 1.6       | It loads into register R1 the value given by contents(contents(100+contents(R2))) and the cost is 3.                                                                                           |   |    |    |
|           | <b>Part B</b>                                                                                                                                                                                  |   |    |    |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |           |           |     |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-----------|-----|
| 2  | <p>Write the different techniques for optimizing basic blocks with examples.</p> <ol style="list-style-type: none"> <li>1. The DAG representation of Basic Blocks</li> <li>2. Finding local common subexpression</li> <li>3. Dead code elimination</li> <li>4. The use of Algebraic Identities</li> <li>5. Representation of array references</li> <li>6. Pointer assignments and procedure calls</li> </ol> <p>Any 5 points with example = <math>2 * 5 = 10</math> marks</p>                                                                                                                                            | 10        | L4        | CO4 |
| 3  | <p>Consider the following source code:</p> <pre> for i from 1 to 10 do   for j from 1 to 10 do     C[i,j] = A[i,j] + B[i,j]; </pre> <p>a) Write the TAC considering the array sizes as 10 X 12 where each element takes 4 bytes of memory space.</p> <p>b) Identify the leaders and form the basic blocks</p> <p>c) Draw the flow graphs and identify the loops if one exists</p> <p>Ans: a) Three address code with properly manipulating the array addresses and jump statements. ---- 4 Marks</p> <p>b) Identifying more at least 6 leaders properly – 3 Marks</p> <p>c) Neatly drawing the flow graphs – 3 Marks</p> | 4+3<br>+3 | L3        | CO3 |
| 4. | <p>Explain the main issues in code generation with examples for each of the issues in detail.</p> <ol style="list-style-type: none"> <li>1. Input to the code generation</li> <li>2. The Target program</li> <li>3. Instruction Selection</li> <li>4. Register allocation</li> <li>5. Evaluation order</li> </ol> <p>Explanation + Examples ----- <math>2 * 5 = 10</math> Marks</p>                                                                                                                                                                                                                                      | 10        | L2        | CO4 |
| 5. | <p>How the following basic block can be optimized?</p> <ol style="list-style-type: none"> <li>1. a=10</li> <li>2. b=4*a</li> <li>3. t1=i*j</li> <li>4. c=t1+b</li> <li>5. t2=15*a</li> <li>6. d=t2*c</li> <li>7. e=i</li> <li>8. t3=e*j</li> <li>9. t4=i*a</li> <li>10. c=t3+t4</li> </ol> <p>Optimized code:</p> <ol style="list-style-type: none"> <li>1. a=10</li> <li>2. b=4*a</li> <li>3. t1=i*j</li> <li>4. c=t1+b</li> <li>5. t2=15*a</li> <li>6. d=t2*c</li> <li>7. t4=i*a</li> <li>8. c=t3+t4</li> </ol>                                                                                                        | 5+5       | L3,<br>L2 | CO4 |

|    |                                                                                                                                                                                                                                              |    |    |     |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|-----|
|    | With proper reason --- 6 Marks<br>Explain with examples Peephole optimization. --- 4 Marks                                                                                                                                                   |    |    |     |
| 6. | <p>Using backpatching manipulate the Truelist and the Falselist at the root node of the annotated parse tree, which should be constructed for the expression: <math>a &lt; b \ \&amp;\&amp; \ b &gt; c \ \&amp;\&amp; \ a \neq d</math>.</p> | 10 | L4 | CO4 |